# Infosys Technical Question Bank

Role: Application Development & Maintenance (Backend / .NET Core)

**JD Focus Area:** "Strong understanding of object-oriented programming concepts and backend development fundamentals." The interviewer will test your foundational knowledge before moving to frameworks.

## 1. Object-Oriented Programming (OOP) & C# Fundamentals

- Explain the difference between Abstraction and Encapsulation with a practical C# example.
- How does method overriding differ from method hiding (using the `new` keyword) in C#?
- Can you instantiate an Abstract class? Why or why not?
- What is the difference between an Interface and an Abstract class in C# 8.0 and above?
- Explain the concept of Polymorphism. How is it implemented in a backend API context?
- What is Dependency Injection, and which SOLID principle does it fulfill?
- Explain the difference between Value Types and Reference Types. Where are they stored in memory?
- What is Boxing and Unboxing, and why is it considered a performance overhead?
- How does the Garbage Collector (GC) work in .NET, and what are GC Generations?
- Explain the use of the `using` statement and its relationship with the `IDisposable` interface.
- What is the difference between `String` and `StringBuilder`? When should you use which?
- How do `ref` and `out` parameters differ in C#?
- Explain Extension Methods. What are the rules for creating them?
- What is the difference between early binding (compile-time) and late binding (runtime)?
- How does C# handle multiple inheritance? (Explain through Interfaces).

## 2. .NET Core & ASP.NET Core Web API

- Explain the request processing pipeline in ASP.NET Core. What is Middleware?
- What is the difference between `app.Use()`, `app.Run()`, and `app.Map()` in the middleware pipeline?
- Explain the three Dependency Injection lifetimes in .NET Core: Transient, Scoped, and Singleton.
- If a Singleton service injects a Transient service, what happens? (Captive Dependency).
- What is Kestrel, and why is it used as the default cross-platform web server?
- How do you handle environment-specific configurations (e.g., appsettings.Development.json)?
- Explain Content Negotiation in ASP.NET Core Web API.
- What are Action Filters, and at what stage of the pipeline do they execute?
- How do you implement global exception handling in a .NET Core Web API?
- What is the difference between `IApplicationBuilder` and `IEndpointRouteBuilder`?
- How do Model Binding and Data Annotations work in ASP.NET Core?
- What is the Options Pattern (`IOptions<T>`) and how is it used to read configuration?

| Service Lifetime | Instance Creation Behavior | Common Use Case |
|---|---|---|
| **Transient** | New instance created every single time it is requested. | Lightweight, stateless services. |
| **Scoped** | One instance created per HTTP request (connection). | Entity Framework `DbContext`. |
| **Singleton** | One instance created for the entire application lifetime. | Caching services, configuration settings. |

## 3. REST APIs & System Integration

- What are the core constraints of a RESTful architecture?
- Explain the difference between HTTP PUT and HTTP PATCH regarding idempotency.
- How do you secure a REST API? Explain JWT (JSON Web Tokens) authentication.
- What is CORS, and how do you configure it in an ASP.NET Core application?
- How do you handle API versioning (Query String, Header, URL Path)?
- What HTTP status code should be returned for a successful POST request that creates a resource?
- Explain the concept of HATEOAS in mature REST APIs.
- How do you consume external third-party APIs from within a .NET Core application (e.g., HttpClientFactory)?
- What are the advantages of using `IHttpClientFactory` over creating new `HttpClient` instances?
- How do you handle rate-limiting and circuit breaking when integrating with external systems?

## 4. Entity Framework Core (EF Core)

- What is Entity Framework Core, and how does it implement the Object-Relational Mapping (ORM) pattern?
- Explain the difference between Code-First and Database-First approaches. Which do you prefer and why?
- What is the purpose of the `DbContext` class in EF Core?
- How do Migrations work? What commands are used to add and apply a migration?
- Explain the difference between Lazy Loading, Eager Loading, and Explicit Loading.
- How do you avoid the "N+1 query problem" in EF Core?
- What is change tracking in EF Core, and how does the `AsNoTracking()` method improve performance?
- How do you handle concurrency conflicts in EF Core (Optimistic Concurrency)?
- What is the Fluent API, and how does it differ from Data Annotations for model configuration?
- How do you execute raw SQL queries or stored procedures using EF Core?
- Explain how to configure a Many-to-Many relationship in EF Core.
- What are Global Query Filters, and how are they used for soft-delete functionality?
- How do you implement transactions in EF Core to ensure data integrity across multiple operations?
- What is Connection Pooling, and how is it managed in EF Core?
- Explain the repository pattern and how it abstracts EF Core logic from controllers.

## 5. SQL Server & Database Concepts

- What are the differences between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?
- Explain the difference between a Clustered Index and a Non-Clustered Index.
- What are the ACID properties of a database transaction?
- What is a Stored Procedure, and how does it differ from a User-Defined Function (UDF)?
- How do you troubleshoot a slow-running SQL query? (Execution plans, indexing).
- What is normalization? Explain the First, Second, and Third Normal Forms (1NF, 2NF, 3NF).
- What is a Trigger, and in what scenarios should it be avoided?
- Explain the difference between `TRUNCATE`, `DELETE`, and `DROP` commands.
- What are deadlocks in SQL Server, and how can you prevent them?
- Explain the difference between a Primary Key and a Unique Constraint.
- What is the role of a Foreign Key, and what happens during cascading deletes?
- How do you write a query to find the second highest salary in an Employee table?

**JD Focus Area:** "Troubleshoot production issues, perform root-cause analysis, and deliver timely fixes... Contribute to improving application performance." You must demonstrate a systematic approach to debugging.

## 6. Debugging, Logging & Performance Tuning

- Describe your step-by-step approach to performing root-cause analysis on a production issue.
- How do you implement structured logging in .NET Core (e.g., using Serilog or NLog)?
- What information is critical to include in an error log for effective debugging?
- How do you debug an application that is consuming high CPU or Memory in a production environment?
- What is a Memory Leak in a managed language like C#, and how do you diagnose it?
- Explain how you would use Application Insights or similar APM tools to trace a slow API request.
- What are the common causes of slow API performance, and how do you resolve them?
- Explain the concept of caching. When would you use In-Memory caching vs. Distributed Caching (Redis)?
- How does asynchronous programming (`async/await`) improve the scalability of a Web API?
- What is thread starvation, and how does improper use of `.Result` or `.Wait()` cause it?
- How do you handle database connection timeouts?
- Describe a time you refactored a legacy module to improve its reliability and maintainability.

## 7. Unit Testing & Code Quality

- What is the Arrange-Act-Assert (AAA) pattern in unit testing?
- Explain the difference between Unit Testing and Integration Testing.
- What mocking frameworks are you familiar with (e.g., Moq, NSubstitute), and how do you use them?
- How do you test a service that relies on Entity Framework Core? (In-memory database vs repository mocking).
- Why is code coverage important, and what metrics do you track?
- What are some common "code smells" you look for during a code review?
- How do you ensure your code handles edge cases and null values gracefully?
- Explain Test-Driven Development (TDD). Have you used it in a real project?
- How do you test asynchronous methods in xUnit or NUnit?
- What constitutes "clean, reusable, and maintainable code" according to engineering best practices?
- How do you handle testing internal or private methods in C#?
- Describe your process for providing constructive feedback during a peer code review.

## 8. Git, Collaboration & Agile Delivery

- Explain the standard Git flow or GitHub flow workflow.
- What is the difference between `git merge` and `git rebase`? When should you avoid rebasing?
- How do you resolve merge conflicts when collaborating with other developers?
- Explain the difference between a local repository, a remote repository, and a working directory.
- What is the purpose of a `.gitignore` file? What typically goes inside it?
- How do you revert a commit that has already been pushed to a remote branch?
- Describe a typical Sprint in an Agile methodology. What are the key ceremonies?
- How do you estimate the effort required to deliver a new backend feature?
- What is your approach to collaborating with QA engineers to ensure fixes are release-ready?
- Why is maintaining technical documentation important, and what tools do you use (e.g., Swagger/OpenAPI)?
- How do you communicate technical blockers to stakeholders or project managers?
- What does "Definition of Done" (DoD) mean in your current team?
- How do you manage your time when assigned independent modules alongside team code reviews?
- Explain how CI/CD pipelines (like Azure DevOps or GitHub Actions) integrate with Git.
- Describe a scenario where a deployment failed in production. What steps did you take to roll back and fix it?